Network Packet Analyzer (Mini Wireshark)

Complete Technical Report with Full Code and Line-by-Line Explanation

Ansh Prem

Contents

1 Introduction **3**
1.1 Why this project matters . . . 3

2 Core Features **3**

3 System Architecture **4**
3.1 High-level pipeline . . . 4
3.2 Concurrency model (why it is designed this way) . . . 4

4 Folder and File Structure **4**
4.1 Folder tree . . . 4
4.2 What each folder contains . . . 5

5 How To Run (Quick + Full) **5**
5.1 Install dependencies (Linux/Ubuntu/WSL) . . . 5
5.2 Build . . . 5
5.3 Run (examples) . . . 5
5.4 Generate traffic to test . . . 5
5.5 Open PCAP in Wireshark . . . 6

6 Algorithms (Flow Hashing and Top-N Selection) **6**
6.1 Flow Key Construction (Canonical 5-tuple) . . . 6
6.2 Flow Hashing (unordered_map key hashing) . . . 6
6.3 Top-N Selection (Efficient ranking) . . . 7
6.4 Pseudocode Summary . . . 7

7 FULL CODE WITH LINE-BY-LINE EXPLANATION **8**
7.1 CMakeLists.txt (Build System) . . . 8
7.2 include/npa/config.hpp (CLI Configuration) . . . 8
7.3 src/config.cpp (CLI Parsing with Boost.Program_options) . . . 9
7.4 include/npa/packet.hpp (PacketInfo Model) . . . 11
7.5 include/npa/util.hpp and src/util.cpp (Byte Helpers) . . . 12
7.6 include/npa/pcap$_capture.hppandsrc/pcap_capture.cpp(CaptureLayer)$ . . . 13
7.7 include/npa/pcap$_writer.hppandsrc/pcap_writer.cpp(PCAPExport)$ . . . 15
7.8 include/npa/parser.hpp and src/parser.cpp (Protocol Parsing + HTTP Host Extraction) . . . 16
7.9 include/npa/analytics.hpp and src/analytics.cpp (Flows, Talkers, HTTP Hosts) . . . 20

7.10 include/npa/stats.hpp and src/stats.cpp (Counters + Reporting) 23
7.11 src/main.cpp (Orchestration, Threads, Timers, Pipeline) 26

8 Extra Diagrams for Interviews **28**
8.1 Call Graph 28
8.2 Dependency Map 29

9 Notes and Limitations (Honest Engineering) **29**

10 Future Work (Resume-Heavy Extensions) **29**

11 Conclusion **29**

1 Introduction

This document is a complete explanation report for a **Network Packet Analyzer** implemented in **C++17**. The tool captures packets from a real network interface, parses protocol headers, performs traffic analytics (**Top Talkers**, **Flow Tracking**, **HTTP Host extraction**), and optionally saves captured traffic to a Wireshark-compatible **.pcap** file.

1.1 Why this project matters

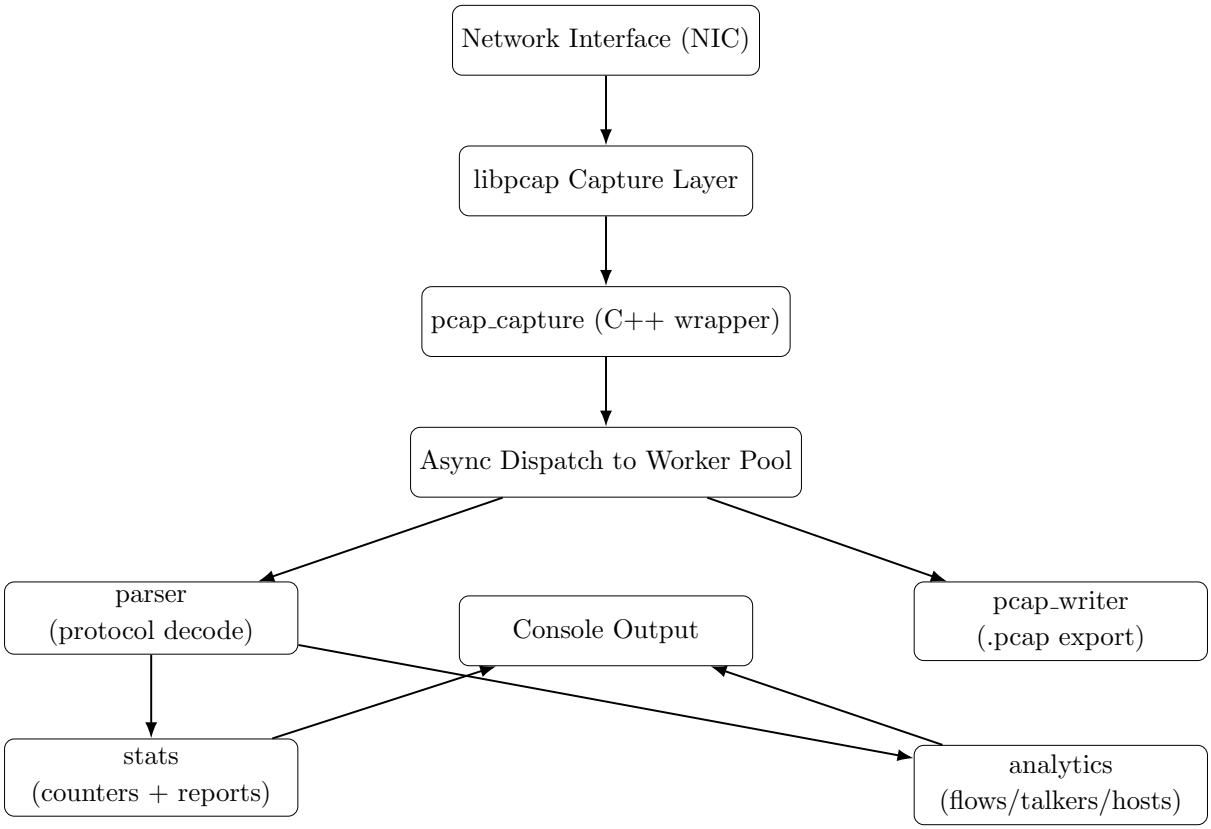
Networks are the backbone of modern systems. Debugging latency, diagnosing failures, monitoring bandwidth usage, and doing security analysis all require visibility into network traffic. Tools like Wireshark are powerful but complex. This project demonstrates the **core engineering concepts** behind such tools: packet capture, protocol parsing, concurrent processing, real-time analytics, and exportable artifacts.

2 Core Features

- **Live packet capture** using libpcap (Linux/macOS) / Npcap (Windows).
- **Protocol parsing**: Ethernet → IPv4 → TCP/UDP.
- **Real-time statistics**: packet counts, bytes, protocol split.
- **Top Talkers**: top source and destination IPs by bytes.
- **Flow tracking**: 5-tuple (canonicalized) conversation tracking.
- **HTTP Host extraction**: parses plaintext HTTP requests to extract Host header.
- **PCAP export**: saves packets to `capture.pcap` readable in Wireshark.
- **Async + multithreaded design**: capture thread + Boost.Asio worker pool.

3 System Architecture

3.1 High-level pipeline



3.2 Concurrency model (why it is designed this way)

- **Capture thread**: continuously pulls packets from libpcap, ensuring we do not miss packets.
- **Worker pool (Boost.Asio)**: CPU-heavy parsing and analytics are executed in parallel.
- **Reason**: capture should never block on parsing; otherwise packet loss increases.

4 Folder and File Structure

4.1 Folder tree

```
network-packet-analyzer/
 include/npa/
    analytics.hpp
    config.hpp
    packet.hpp
    parser.hpp
    pcap_capture.hpp
    pcap_writer.hpp
    stats.hpp
```

```
    util.hpp
 src/
    analytics.cpp
    config.cpp
    main.cpp
    parser.cpp
    pcap_capture.cpp
    pcap_writer.cpp
    stats.cpp
    util.cpp
 CMakeLists.txt
 README.md
```

4.2 What each folder contains

- **include/npa/**: Header files (.hpp) that define interfaces, structs, and class APIs.
- **src/**: Implementation files (.cpp) that implement the actual logic.
- **CMakeLists.txt**: Build instructions.
- **README.md**: User documentation (how to build/run + feature summary).

5 How To Run (Quick + Full)

5.1 Install dependencies (Linux/Ubuntu/WSL)

```
sudo apt update
sudo apt install build-essential cmake libboost-all-dev libpcap-dev
```

5.2 Build

```
mkdir build
cd build
cmake ..
make -j
```

5.3 Run (examples)

```
sudo ./npa --iface eth0
sudo ./npa --iface eth0 --filter "tcp" -v
sudo ./npa --iface eth0 --pcap-out capture.pcap
sudo ./npa --iface eth0 --top-talkers 5 --top-flows 5 --top-hosts 5 -v
```

5.4 Generate traffic to test

```
curl http://example.com
curl https://google.com
ping google.com
```

5.5 Open PCAP in Wireshark

```
wireshark capture.pcap
```

6 Algorithms (Flow Hashing and Top-N Selection)

This section explains the two main algorithmic components that make the analyzer **Wireshark-like**: **flow hashing** (for flow tracking) and **Top-N selection** (for top talkers/flows/hosts).

6.1 Flow Key Construction (Canonical 5-tuple)

A network **flow** is identified by:

$$(\text{srcIP}, \text{dstIP}, \text{srcPort}, \text{dstPort}, \text{protocol})$$

Traffic is bi-directional, and we want:

$$A \to B \quad \text{and} \quad B \to A$$

to map to the **same** flow entry.

Canonicalization idea:

- Create endpoints $E_1 = (\text{srcIP}, \text{srcPort})$ and $E_2 = (\text{dstIP}, \text{dstPort})$.
- Sort endpoints: if $E_2 < E_1$, swap them.
- Final key becomes (proto, E_a, E_b).

Why this matters: it ensures the flow table does not store two entries for the same conversation.

6.2 Flow Hashing (unordered_map key hashing)

Flows are stored in:

```
unordered_map<FlowKey, FlowEntry, FlowKeyHash>
```

A hash is computed by combining:

- protocol tag (TCP/UDP)
- endpoint A IP string
- endpoint A port
- endpoint B IP string
- endpoint B port

Complexity:

- insertion/lookup average case: $O(1)$
- update per packet average case: $O(1)$

6.3 Top-N Selection (Efficient ranking)

For top talkers / top flows / top hosts:

- We maintain hash maps that accumulate counters (bytes/hits).
- When printing, we compute the top N entries efficiently.

Approach used:

- Convert map entries into a vector.
- Use `std::nth_element` to select the first N biggest values.
- Resize vector to N.
- Sort the remaining N entries for clean output ordering.

Why not full sort always?

- Sorting all entries is $O(M \log M)$ where M can grow large.
- `nth_element` reduces work to about $O(M)$ average, then sorting only N items costs $O(N \log N)$.

Complexity summary:

$$O(M) + O(N \log N)$$

6.4 Pseudocode Summary

Listing 1: Pseudocode: Flow Tracking + Top-N

```
# Flow Tracking (per packet)
function ingest(packet):
    key = canonical_flow_key(packet.proto, packet.srcIP:srcPort, packet.dstIP:dstPort)
    flow[key].packets += 1
    flow[key].bytes += packet.wirelen
    flow[key].lastSeen = now

# Top-N Selection
function topN(map, N):
    vec = entries(map)
    nth_element(vec, N, by value descending)
    vec.resize(N)
    sort(vec, by value descending)
    return vec
```

7 FULL CODE WITH LINE-BY-LINE EXPLANATION

7.1 CMakeLists.txt (Build System)

Purpose: Defines how to compile and link the project with Boost and libpcap.

Listing 2: CMakeLists.txt (annotated)

```
cmake_minimum_required(VERSION 3.16)                    # Require a modern CMake version for consistent behavior
project(npa LANGUAGES CXX)                              # Define project name and language (C++)

set(CMAKE_CXX_STANDARD 17)                              # Use C++17 for std::optional, string_view, etc.
set(CMAKE_CXX_STANDARD_REQUIRED ON)                     # Fail build if compiler cannot do C++17

find_package(Boost REQUIRED COMPONENTS program_options)# Find Boost and specifically Program_options for CLI parsing
find_library(PCAP_LIB pcap REQUIRED)                    # Find libpcap library (packet capture backend)

add_executable(npa                                        # Create executable named npa
  src/main.cpp                                            # App orchestration (threads, timers, pipeline)
  src/config.cpp                                          # CLI parsing implementation
  src/util.cpp                                            # Byte helpers (endianness, ip formatting)
  src/pcap_capture.cpp                                    # Packet capture wrapper over libpcap
  src/parser.cpp                                          # Protocol parsing + HTTP Host extraction
  src/stats.cpp                                           # Counters + reporting
  src/pcap_writer.cpp                                     # PCAP export writer
  src/analytics.cpp                                       # Top talkers, top flows, top hosts
)

target_include_directories(npa PRIVATE include)        # Allow includes like npa/parser.hpp from include/
target_link_libraries(npa PRIVATE Boost::program_options ${PCAP_LIB}) # Link Boost.Program_options + libpcap
```

7.2 include/npa/config.hpp (CLI Configuration)

Purpose: Stores command-line settings in a single struct so all modules can use the same configuration.

Listing 3: include/npa/config.hpp (annotated)

```
#pragma once                                          // Prevent multiple inclusion of this header
#include <string>                                     // Needed for std::string

```

```
namespace npa {                                         // Project namespace to avoid name conflicts

struct Config {                                         // Holds all CLI parameters (single source of truth)
  std::string iface;                                    // Network interface name (eth0, wlan0, etc.)
  std::string filter;                                   // BPF filter string (e.g., tcp or udp)
  bool promisc = true;                                  // Enable promiscuous mode to see more traffic
  int snaplen = 65535;                                  // Max bytes captured per packet (full frame)
  int stats_every_sec = 1;                              // Print stats every N seconds
  bool verbose = false;                                 // If true, print per-packet decoded output

  std::string pcap_out;                                 // Output .pcap filename; empty means disabled
  int top_talkers = 5;                                  // Top N talkers (0 disables)
  int top_flows = 5;                                    // Top N flows (0 disables)
  int top_hosts = 5;                                    // Top N HTTP hosts (0 disables)
};

Config parse_args(int argc, char** argv);               // Parse CLI into Config; exits on --help

} // namespace npa                                      // End namespace
```

7.3 src/config.cpp (CLI Parsing with Boost.Program_options)

Purpose: Converts `argv` into `Config`. This keeps main.cpp clean and professional.

Listing 4: src/config.cpp (annotated)

```
#include npa/config.hpp                                // Import Config struct + parse_args declaration
#include <boost/program_options.hpp>                    // Boost CLI parsing library
#include <iostream>                                     // For printing help text

namespace npa {                                         // Keep everything under npa namespace

Config parse_args(int argc, char** argv) {              // Entry function to parse command line
  namespace po = boost::program_options;                // Alias for shorter code

  Config cfg;                                           // Create config object to fill from CLI
  po::options_description desc(Network Packet Analyzer options); // Help/usage text container

  desc.add_options()                                    // Define allowed CLI options
```

```
    (help,h, Show help)                          // --help or -h prints help and exits
    (iface,i, po::value<std::string>(&cfg.iface)->default_value(eth0),
      Interface to capture on)                   // Set interface name, default eth0
    (filter,f, po::value<std::string>(&cfg.filter)->default_value(tcp or udp),
      BPF filter)                                // Set BPF filter expression
    (promisc, po::value<bool>(&cfg.promisc)->default_value(true),
      Promiscuous mode (true/false))             // Toggle promiscuous capture
    (snaplen, po::value<int>(&cfg.snaplen)->default_value(65535),
      Snap length)                               // How many bytes of each packet to capture
    (stats-every, po::value<int>(&cfg.stats_every_sec)->default_value(1),
      Print stats every N seconds)               // Reporting interval
    (verbose,v, po::bool_switch(&cfg.verbose),
      Verbose output)                            // -v prints each packet summary

    (pcap-out, po::value<std::string>(&cfg.pcap_out)->default_value(),
      Write captured packets to this .pcap file) // Enable PCAP output if non-empty
    (top-talkers, po::value<int>(&cfg.top_talkers)->default_value(5),
      Print top N SRC/DST talkers (0 disables))  // Top talkers by bytes
    (top-flows, po::value<int>(&cfg.top_flows)->default_value(5),
      Print top N flows (0 disables))            // Top flows by bytes
    (top-hosts, po::value<int>(&cfg.top_hosts)->default_value(5),
      Print top N HTTP hosts (0 disables));      // Top HTTP Host headers by hits

  po::variables_map vm;                          // Map to hold parsed values
  po::store(po::parse_command_line(argc, argv, desc), vm); // Parse argv into vm
  po::notify(vm);                                // Write parsed values into cfg fields

  if (vm.count(help)) {                          // If user requested help
    std::cout << desc << \n;                     // Print help text
    std::exit(0);                                // Exit program normally
  }

  if (cfg.stats_every_sec <= 0) cfg.stats_every_sec = 1;// Guard: avoid zero/negative timers
  if (cfg.snaplen <= 0) cfg.snaplen = 65535;     // Guard: invalid snaplen fallback
  if (cfg.top_talkers < 0) cfg.top_talkers = 0;  // Guard: negative disables
  if (cfg.top_flows < 0) cfg.top_flows = 0;      // Guard: negative disables
  if (cfg.top_hosts < 0) cfg.top_hosts = 0;      // Guard: negative disables

  return cfg;                                    // Return final validated configuration
}

} // namespace npa                               // End namespace
```

7.4 include/npa/packet.hpp (PacketInfo Model)

Purpose: A single struct that represents **parsed** packet fields used by Stats and Analytics.

Listing 5: include/npa/packet.hpp (annotated)

```cpp
#pragma once                                        // Prevent multiple inclusion
#include <cstdint>                                  // Fixed-width integer types (uint16_t etc.)
#include <string>                                   // std::string for IPs, HTTP fields

namespace npa {                                     // Project namespace

enum class L4Proto { TCP, UDP, OTHER };             // Layer-4 protocol type used in reporting

struct PacketInfo {                                 // Parsed packet metadata structure
  bool ipv4 = false;                                // True if packet contains IPv4 header
  L4Proto l4 = L4Proto::OTHER;                       // Which L4 protocol was parsed

  std::string src_ip;                               // Source IPv4 dotted string
  std::string dst_ip;                               // Destination IPv4 dotted string

  uint16_t src_port = 0;                            // TCP/UDP source port (0 if unknown)
  uint16_t dst_port = 0;                            // TCP/UDP dest port (0 if unknown)

  uint16_t ip_total_len = 0;                        // IP total length field (helps debugging)
  uint8_t  ip_ttl = 0;                              // IP TTL field (useful for diagnostics)

  bool http = false;                                // True if an HTTP request was detected
  std::string http_method;                          // HTTP method e.g., GET, POST
  std::string http_uri;                             // HTTP URI path
  std::string http_host;                            // HTTP Host header value

  uint32_t caplen = 0;                              // Captured bytes in this packet (pcap caplen)
  uint32_t wirelen = 0;                             // Actual on-wire size (pcap len)
};

} // namespace npa                                  // End namespace
```

7.5 include/npa/util.hpp and src/util.cpp (Byte Helpers)

Purpose: Networking protocols use network byte order (big-endian). These helpers read big-endian fields and format IPv4.

Listing 6: include/npa/util.hpp (annotated)

```
#pragma once                                        // Header include guard
#include <cstdint>                                  // For uint16_t/uint32_t
#include <string>                                   // For std::string

namespace npa {                                     // Project namespace

uint16_t read_be16(const uint8_t* p);               // Read 16-bit big-endian from raw bytes
uint32_t read_be32(const uint8_t* p);               // Read 32-bit big-endian from raw bytes
std::string ipv4_to_string(uint32_t be);            // Convert IPv4 (big-endian) to x.x.x.x

} // namespace npa
```

Listing 7: src/util.cpp (annotated)

```
#include npa/util.hpp                             // Import util declarations
#include <sstream>                                  // For building string representation

namespace npa {

uint16_t read_be16(const uint8_t* p) {              // Read 2 bytes big-endian
  return (uint16_t(p[0]) << 8) | uint16_t(p[1]);    // Combine bytes: p0 high, p1 low
}

uint32_t read_be32(const uint8_t* p) {              // Read 4 bytes big-endian
  return (uint32_t(p[0]) << 24) |                   // Byte0 highest-order
         (uint32_t(p[1]) << 16) |                   // Byte1
         (uint32_t(p[2]) << 8)  |                   // Byte2
          uint32_t(p[3]);                           // Byte3 lowest-order
}

std::string ipv4_to_string(uint32_t be) {           // Convert IPv4 integer to dotted string
  uint8_t a = (be >> 24) & 0xFF;                    // Extract highest byte
  uint8_t b = (be >> 16) & 0xFF;                    // Extract next byte
  uint8_t c = (be >> 8)  & 0xFF;                    // Extract next byte
  uint8_t d = (be)       & 0xFF;                    // Extract lowest byte
  std::ostringstream oss;                            // Build output string
  oss << int(a) << . << int(b) << . << int(c) << . << int(d); // Dotted IPv4
  return oss.str();                                  // Return final string
}

} // namespace npa
```

7.6 include/npa/pcap$_c$apture.hppandsrc/pcap$_c$apture.cpp(CaptureLayer)

Purpose: Wrap libpcap so capture is isolated from application logic.

Listing 8: include/npa/pcap$_c$apture.hpp(annotated)

```cpp
#pragma once                                          // Prevent multiple inclusion
#include <cstdint>                                    // For uint32_t
#include <functional>                                 // For std::function callback
#include <string>                                     // For std::string iface/filter

namespace npa {

struct PcapPacket {                                   // One captured packet delivered to callback
  const uint8_t* data;                                // Raw bytes pointer (valid only during callback)
  uint32_t caplen;                                    // Captured length
  uint32_t wirelen;                                   // On-wire length
  const void* pcap_hdr;                               // Pointer to pcap_pkthdr (timestamp + lengths)
};

class PcapCapture {                                   // C++ wrapper around pcap_t*
public:
  using Callback = std::function<void(const PcapPacket&)>; // Callback signature

  PcapCapture() = default;                            // Default constructor
  ~PcapCapture();                                     // Destructor closes pcap handle safely

  void run(const std::string& iface,                  // Start capture on an interface
           const std::string& bpf_filter,             // Apply BPF filter if provided
           bool promisc,                              // Promiscuous mode toggle
           int snaplen,                               // Snapshot length
           Callback cb);                              // Callback invoked per packet

  void stop();                                        // Stop capture loop cleanly

  void* native_handle() const { return handle_; }     // Expose raw pcap_t* for pcap_writer

private:
  void* handle_ = nullptr;                            // Stores pcap_t* as void*
  bool stop_ = false;                                 // Loop stop flag
};

} // namespace npa
```

Listing 9: src/pcap$_c$apture.cpp(annotated)

```cpp
#include npa/pcap_capture.hpp                       // Our wrapper API
#include <pcap/pcap.h>                                // libpcap C API
#include <stdexcept>                                  // For std::runtime_error
#include <string>                                     // For std::string
```

```

namespace npa {

struct DispatchUser {                                   // Context passed to libpcap callback
  PcapCapture::Callback* cb;                            // Pointer to callback
};

static void pcap_dispatch_cb(u_char* user,              // libpcap callback signature
                             const struct pcap_pkthdr* h,
                             const u_char* bytes) {
  auto* u = reinterpret_cast<DispatchUser*>(user);      // Recover context
  PcapPacket p{                                         // Wrap libpcap parameters
    reinterpret_cast<const uint8_t*>(bytes),            // Raw bytes
    static_cast<uint32_t>(h->caplen),                   // Captured length
    static_cast<uint32_t>(h->len),                      // On-wire length
    h                                                   // Header pointer (timestamp)
  };
  (*(u->cb))(p);                                        // Call user callback
}

PcapCapture::~PcapCapture() {                           // Destructor
  if (handle_) {                                        // If handle exists
    pcap_close(reinterpret_cast<pcap_t*>(handle_));     // Close libpcap handle
    handle_ = nullptr;                                  // Reset pointer
  }
}

void PcapCapture::run(const std::string& iface,
                      const std::string& bpf_filter,
                      bool promisc,
                      int snaplen,
                      Callback cb) {
  char errbuf[PCAP_ERRBUF_SIZE]{0};                     // Error buffer

  pcap_t* p = pcap_open_live(                           // Open interface
      iface.c_str(),                                    // Interface name
      snaplen,                                          // Snapshot length
      promisc ? 1 : 0,                                  // Promiscuous
      1000,                                             // Timeout (ms)
      errbuf);                                          // Error output
  if (!p) throw std::runtime_error(std::string(pcap_open_live failed: ) + errbuf);

  handle_ = p;                                          // Store handle

  bpf_program fp{};                                     // Compiled filter program
  if (!bpf_filter.empty()) {                            // Only compile if non-empty
    if (pcap_compile(p, &fp, bpf_filter.c_str(), 1, PCAP_NETMASK_UNKNOWN) < 0) {
      throw std::runtime_error(std::string(pcap_compile failed: ) + pcap_geterr(p));
    }
    if (pcap_setfilter(p, &fp) < 0) {                   // Install filter
      pcap_freecode(&fp);
      throw std::runtime_error(std::string(pcap_setfilter failed: ) + pcap_geterr(p));
    }
```

```
    pcap_freecode(&fp);                                  // Free filter after install
  }

  stop_ = false;                                         // Reset stop flag
  DispatchUser u{&cb};                                   // Provide callback pointer

  while (!stop_) {                                       // Main capture loop
    int rc = pcap_dispatch(p, 64, pcap_dispatch_cb, reinterpret_cast<u_char*>(&u)); // Process batch
    if (rc < 0) break;                                   // Error or breakloop
  }
}

void PcapCapture::stop() {                               // Stop capture
  stop_ = true;                                          // Set stop flag
  if (handle_) {                                         // If handle exists
    pcap_breakloop(reinterpret_cast<pcap_t*>(handle_)); // Force dispatch to return
  }
}

} // namespace npa
```

7.7 include/npa/pcap$_w$riter.hppandsrc/pcap$_w$riter.cpp(PCAPExport)

Purpose: Save packets to a `.pcap` file for offline analysis in Wireshark.

Listing 10: include/npa/pcap$_w$riter.hpp(*annotated*)

```
#pragma once                                             // Header guard
#include <pcap/pcap.h>                                   // pcap types + dump API
#include <string>                                        // Filename storage

namespace npa {

class PcapWriter {                                       // Writes packets to a .pcap file
public:
  PcapWriter() = default;                                // Default constructor
  ~PcapWriter();                                         // Destructor closes file if open

  void open(pcap_t* handle, const std::string& filename); // Open dumper with capture handle (link type)
  void write(const pcap_pkthdr* hdr, const u_char* bytes); // Write one packet record
  void close();                                          // Close dumper handle

  bool enabled() const { return dumper_ != nullptr; }  // True if writer is active
private:
  pcap_dumper_t* dumper_ = nullptr;                      // Dump stream pointer
};

} // namespace npa
```

Listing 11: src/pcap$_w$*riter.cpp*(*annotated*)

```cpp
#include npa/pcap_writer.hpp                          // Class declaration
#include <stdexcept>                                   // Exceptions on failure

namespace npa {

PcapWriter::~PcapWriter() { close(); }                 // RAII cleanup

void PcapWriter::open(pcap_t* handle, const std::string& filename) {
  if (filename.empty()) return;                        // Feature disabled
  if (!handle) throw std::runtime_error(PcapWriter::open: null pcap handle);

  dumper_ = pcap_dump_open(handle, filename.c_str());  // Open output pcap file
  if (!dumper_) throw std::runtime_error(pcap_dump_open failed (check path/permissions
    ↪ ));
}

void PcapWriter::write(const pcap_pkthdr* hdr, const u_char* bytes) {
  if (!dumper_ || !hdr || !bytes) return;              // Safety guard
  pcap_dump(reinterpret_cast<u_char*>(dumper_), hdr, bytes); // Write record
}

void PcapWriter::close() {
  if (dumper_) {                                       // If open
    pcap_dump_close(dumper_);                          // Close + flush
    dumper_ = nullptr;                                 // Reset
  }
}

} // namespace npa
```

7.8 include/npa/parser.hpp and src/parser.cpp (Protocol Parsing + HTTP Host Extraction)

Purpose: Convert raw bytes into structured protocol metadata (IP addresses, ports, L4 protocol) and extract HTTP Host from plaintext HTTP.

Listing 12: include/npa/parser.hpp (annotated)

```cpp
#pragma once                                           // Header guard
#include npa/packet.hpp                               // PacketInfo definition
#include <cstdint>                                     // For uint32_t
#include <optional>                                    // std::optional return type

namespace npa {

std::optional<PacketInfo> parse_packet(                // Parse Ethernet->IPv4->TCP/
    ↪ UDP (+HTTP)
    const uint8_t* data,                               // Raw bytes
    uint32_t caplen,                                   // Captured length
    uint32_t wirelen);                                 // On-wire length

} // namespace npa
```

Listing 13: src/parser.cpp (annotated)

```cpp
#include npa/parser.hpp                                // parse_packet()
#include npa/util.hpp                                  // endianness + ip formatting
#include <algorithm>                                      // std::min
#include <cctype>                                         // std::tolower, std::isspace
#include <string_view>                                    // efficient string-like view

namespace npa {

static constexpr uint32_t ETH_HDR = 14;                   // Ethernet header bytes
static constexpr uint16_t ETHERTYPE_IPV4 = 0x0800;         // Ethernet type for IPv4

static bool starts_with(std::string_view s, std::string_view p) {
  return s.size() >= p.size() && s.substr(0, p.size()) == p; // prefix check
}

static std::string trim(std::string s) {                  // whitespace trim for header value cleanup
  auto not_space = [](unsigned char c){ return !std::isspace(c); };
  while (!s.empty() && !not_space((unsigned char)s.front())) s.erase(s.begin());
  while (!s.empty() && !not_space((unsigned char)s.back())) s.pop_back();
  return s;
}

static bool ieq_prefix(std::string_view s, std::string_view p) {
  if (s.size() < p.size()) return false;
  for (size_t i = 0; i < p.size(); i++) {
    if (std::tolower((unsigned char)s[i]) != std::tolower((unsigned char)p[i])) return false;
  }
  return true;
}

static void try_parse_http(PacketInfo& info, const uint8_t* payload, uint32_t payload_len) {
  if (!payload || payload_len < 16) return;

  uint32_t n = std::min<uint32_t>(payload_len, 4096);  // limit scanning
  std::string_view sv(reinterpret_cast<const char*>(payload), n);

  const char* methods[] = {GET , POST , HEAD , PUT , DELETE , OPTIONS , PATCH };
  bool is_http = false;
  std::string method;

  for (auto m : methods) {
    if (starts_with(sv, m)) {
      is_http = true;
      method = std::string(m);
      method.pop_back();                                  // remove trailing space
      break;
    }
  }
  if (!is_http) return;

```

```
  auto line_end = sv.find(\r\n);
  if (line_end == std::string_view::npos) return;

  std::string_view line = sv.substr(0, line_end);
  size_t sp1 = line.find(' ');
  if (sp1 == std::string_view::npos) return;
  size_t sp2 = line.find(' ', sp1 + 1);
  if (sp2 == std::string_view::npos) return;

  std::string uri(line.substr(sp1 + 1, sp2 - (sp1 + 1)));

  std::string host;
  size_t pos = line_end + 2;

  while (pos < sv.size()) {
    size_t next = sv.find(\r\n, pos);
    if (next == std::string_view::npos) break;
    if (next == pos) break;                              // end headers

    std::string_view hline = sv.substr(pos, next - pos);
    if (ieq_prefix(hline, Host:)) {
      std::string v(hline.substr(5));
      host = trim(v);
      break;
    }
    pos = next + 2;
  }

  info.http = true;
  info.http_method = method;
  info.http_uri = uri;
  info.http_host = host;
}

std::optional<PacketInfo> parse_packet(const uint8_t* data, uint32_t caplen, uint32_t
    ↪ wirelen) {
  PacketInfo info;
  info.caplen = caplen;
  info.wirelen = wirelen;

  if (!data || caplen < ETH_HDR) return std::nullopt;

  uint16_t etherType = read_be16(data + 12);
  if (etherType != ETHERTYPE_IPV4) return std::nullopt;

  const uint8_t* ip = data + ETH_HDR;
  uint32_t ip_avail = caplen - ETH_HDR;
  if (ip_avail < 20) return std::nullopt;

  uint8_t ver_ihl = ip[0];
  uint8_t ver = ver_ihl >> 4;
  uint8_t ihl = (ver_ihl & 0x0F) * 4;
  if (ver != 4 || ihl < 20) return std::nullopt;
  if (ip_avail < ihl) return std::nullopt;
```

```

  info.ipv4 = true;
  info.ip_total_len = read_be16(ip + 2);
  info.ip_ttl = ip[8];
  uint8_t proto = ip[9];

  uint32_t src_be = read_be32(ip + 12);
  uint32_t dst_be = read_be32(ip + 16);
  info.src_ip = ipv4_to_string(src_be);
  info.dst_ip = ipv4_to_string(dst_be);

  const uint8_t* l4 = ip + ihl;
  uint32_t l4_avail = ip_avail - ihl;

  if (proto == 6) {
    if (l4_avail < 20) return info;
    info.l4 = L4Proto::TCP;

    info.src_port = read_be16(l4 + 0);
    info.dst_port = read_be16(l4 + 2);

    uint8_t data_offset = (l4[12] >> 4) * 4;
    if (data_offset < 20 || l4_avail < data_offset) return info;

    const uint8_t* payload = l4 + data_offset;
    uint32_t payload_len = l4_avail - data_offset;

    try_parse_http(info, payload, payload_len);
    if (!info.http) {
      info.http_method.clear();
      info.http_uri.clear();
      info.http_host.clear();
    }
    return info;

  } else if (proto == 17) {
    if (l4_avail < 8) return info;
    info.l4 = L4Proto::UDP;
    info.src_port = read_be16(l4 + 0);
    info.dst_port = read_be16(l4 + 2);
    return info;

  } else {
    info.l4 = L4Proto::OTHER;
    return info;
  }
}

} // namespace npa
```

7.9 include/npa/analytics.hpp and src/analytics.cpp (Flows, Talkers, HTTP Hosts)

Purpose: Maintain streaming analytics: top talkers (bytes/IP), top flows (bytes/5-tuple), top HTTP hosts (hits).

Listing 14: include/npa/analytics.hpp (annotated)

```cpp
#pragma once
#include npa/packet.hpp
#include <cstdint>
#include <string>
#include <unordered_map>
#include <vector>

namespace npa {

struct TalkerEntry {
  std::string ip;
  uint64_t bytes = 0;
};

struct HostEntry {
  std::string host;
  uint64_t hits = 0;
};

enum class FlowProto : uint8_t { TCP = 6, UDP = 17, OTHER = 0 };

struct Endpoint {
  std::string ip;
  uint16_t port = 0;
  bool operator<(const Endpoint& o) const {
    if (ip != o.ip) return ip < o.ip;
    return port < o.port;
  }
};

struct FlowKey {
  FlowProto proto;
  Endpoint a;
  Endpoint b;
  bool operator==(const FlowKey& o) const {
    return proto == o.proto &&
           a.ip == o.a.ip && a.port == o.a.port &&
           b.ip == o.b.ip && b.port == o.b.port;
  }
};

struct FlowKeyHash { std::size_t operator()(const FlowKey& k) const; };

struct FlowEntry {
  FlowKey key;
  uint64_t packets = 0;
  uint64_t bytes = 0;
```

```
  uint64_t first_seen_ms = 0;
  uint64_t last_seen_ms = 0;
};

class Analytics {
public:
  void ingest(const PacketInfo& p, uint64_t ts_ms);

  std::vector<TalkerEntry> top_src_talkers(int N) const;
  std::vector<TalkerEntry> top_dst_talkers(int N) const;
  std::vector<FlowEntry> top_flows(int N) const;
  std::vector<HostEntry> top_http_hosts(int N) const;

private:
  std::unordered_map<std::string, uint64_t> src_bytes_;
  std::unordered_map<std::string, uint64_t> dst_bytes_;
  std::unordered_map<FlowKey, FlowEntry, FlowKeyHash> flows_;
  std::unordered_map<std::string, uint64_t> http_host_hits_;

  static FlowKey make_flow_key(const PacketInfo& p);
};

} // namespace npa
```

Listing 15: src/analytics.cpp (annotated)

```
#include npa/analytics.hpp
#include <algorithm>
#include <functional>

namespace npa {

static std::size_t hcombine(std::size_t h1, std::size_t h2) {
  return h1 ^ (h2 + 0x9e3779b97f4a7c15ULL + (h1 << 6) + (h1 >> 2));
}

std::size_t FlowKeyHash::operator()(const FlowKey& k) const {
  std::hash<std::string> hs;
  std::hash<uint16_t> hp;
  std::hash<uint8_t> hb;
  std::size_t h = 0;
  h = hcombine(h, hb(static_cast<uint8_t>(k.proto)));
  h = hcombine(h, hs(k.a.ip));
  h = hcombine(h, hp(k.a.port));
  h = hcombine(h, hs(k.b.ip));
  h = hcombine(h, hp(k.b.port));
  return h;
}

FlowKey Analytics::make_flow_key(const PacketInfo& p) {
  FlowProto fp = FlowProto::OTHER;
  if (p.l4 == L4Proto::TCP) fp = FlowProto::TCP;
  else if (p.l4 == L4Proto::UDP) fp = FlowProto::UDP;

```

```
  Endpoint e1{p.src_ip, p.src_port};
  Endpoint e2{p.dst_ip, p.dst_port};
  if (e2 < e1) std::swap(e1, e2);
  return FlowKey{fp, e1, e2};
}

static std::vector<TalkerEntry> topN_talkers(const std::unordered_map<std::string,
    ↪ uint64_t>& mp, int N) {
  if (N <= 0) return {};
  std::vector<TalkerEntry> v;
  v.reserve(mp.size());
  for (const auto& kv : mp) v.push_back({kv.first, kv.second});
  if ((int)v.size() > N) {
    std::nth_element(v.begin(), v.begin() + N, v.end(),
      [](const TalkerEntry& a, const TalkerEntry& b){ return a.bytes > b.bytes; });
    v.resize(N);
  }
  std::sort(v.begin(), v.end(),
    [](auto& a, auto& b){ return a.bytes > b.bytes; });
  return v;
}

static std::vector<HostEntry> topN_hosts(const std::unordered_map<std::string,
    ↪ uint64_t>& mp, int N) {
  if (N <= 0) return {};
  std::vector<HostEntry> v;
  v.reserve(mp.size());
  for (const auto& kv : mp) v.push_back({kv.first, kv.second});
  if ((int)v.size() > N) {
    std::nth_element(v.begin(), v.begin() + N, v.end(),
      [](const HostEntry& a, const HostEntry& b){ return a.hits > b.hits; });
    v.resize(N);
  }
  std::sort(v.begin(), v.end(),
    [](auto& a, auto& b){ return a.hits > b.hits; });
  return v;
}

void Analytics::ingest(const PacketInfo& p, uint64_t ts_ms) {
  if (!p.ipv4) return;

  src_bytes_[p.src_ip] += p.wirelen;
  dst_bytes_[p.dst_ip] += p.wirelen;

  FlowKey key = make_flow_key(p);
  auto it = flows_.find(key);
  if (it == flows_.end()) {
    FlowEntry fe;
    fe.key = key;
    fe.packets = 1;
    fe.bytes = p.wirelen;
    fe.first_seen_ms = ts_ms;
    fe.last_seen_ms = ts_ms;
    flows_.emplace(key, std::move(fe));
```

```
  } else {
    it->second.packets += 1;
    it->second.bytes += p.wirelen;
    it->second.last_seen_ms = ts_ms;
  }

  if (p.http && !p.http_host.empty()) {
    http_host_hits_[p.http_host] += 1;
  }
}

std::vector<TalkerEntry> Analytics::top_src_talkers(int N) const { return topN_talkers
    ↪ (src_bytes_, N); }
std::vector<TalkerEntry> Analytics::top_dst_talkers(int N) const { return topN_talkers
    ↪ (dst_bytes_, N); }

std::vector<FlowEntry> Analytics::top_flows(int N) const {
  if (N <= 0) return {};
  std::vector<FlowEntry> v;
  v.reserve(flows_.size());
  for (const auto& kv : flows_) v.push_back(kv.second);

  if ((int)v.size() > N) {
    std::nth_element(v.begin(), v.begin() + N, v.end(),
      [](const FlowEntry& a, const FlowEntry& b){ return a.bytes > b.bytes; });
    v.resize(N);
  }
  std::sort(v.begin(), v.end(),
    [](auto& a, auto& b){ return a.bytes > b.bytes; });
  return v;
}

std::vector<HostEntry> Analytics::top_http_hosts(int N) const { return topN_hosts(
    ↪ http_host_hits_, N); }

} // namespace npa
```

7.10 include/npa/stats.hpp and src/stats.cpp (Counters + Reporting)

Purpose: Centralize counters and reporting. Protect analytics with a mutex because unordered_map is not thread-safe.

Listing 16: include/npa/stats.hpp (annotated)

```
#pragma once
#include npa/packet.hpp
#include npa/analytics.hpp
#include <atomic>
#include <cstdint>
#include <mutex>
#include <string>

namespace npa {

```

```
class Stats {
public:
  void ingest(const PacketInfo& p, uint64_t ts_ms);
  std::string report_and_reset_interval(int top_talkers, int top_flows, int top_hosts);

private:
  std::atomic<uint64_t> total_packets_{0};
  std::atomic<uint64_t> total_bytes_{0};

  std::atomic<uint64_t> tcp_packets_{0};
  std::atomic<uint64_t> udp_packets_{0};
  std::atomic<uint64_t> other_packets_{0};

  std::atomic<uint64_t> interval_packets_{0};
  std::atomic<uint64_t> interval_bytes_{0};

  std::atomic<uint64_t> http_requests_{0};

  mutable std::mutex mx_;
  Analytics analytics_;
};

} // namespace npa
```

Listing 17: src/stats.cpp (annotated)

```
#include npa/stats.hpp
#include <sstream>

namespace npa {

void Stats::ingest(const PacketInfo& p, uint64_t ts_ms) {
  total_packets_.fetch_add(1, std::memory_order_relaxed);
  total_bytes_.fetch_add(p.wirelen, std::memory_order_relaxed);

  interval_packets_.fetch_add(1, std::memory_order_relaxed);
  interval_bytes_.fetch_add(p.wirelen, std::memory_order_relaxed);

  switch (p.l4) {
    case L4Proto::TCP: tcp_packets_.fetch_add(1, std::memory_order_relaxed); break;
    case L4Proto::UDP: udp_packets_.fetch_add(1, std::memory_order_relaxed); break;
    default: other_packets_.fetch_add(1, std::memory_order_relaxed); break;
  }

  if (p.http) http_requests_.fetch_add(1, std::memory_order_relaxed);

  {
    std::lock_guard<std::mutex> lk(mx_);
    analytics_.ingest(p, ts_ms);
  }
}

std::string Stats::report_and_reset_interval(int top_talkers, int top_flows, int
```

```
    ↪ top_hosts) {
  uint64_t ip = interval_packets_.exchange(0, std::memory_order_relaxed);
  uint64_t ib = interval_bytes_.exchange(0, std::memory_order_relaxed);

  uint64_t tp = total_packets_.load(std::memory_order_relaxed);
  uint64_t tb = total_bytes_.load(std::memory_order_relaxed);

  uint64_t tcp = tcp_packets_.load(std::memory_order_relaxed);
  uint64_t udp = udp_packets_.load(std::memory_order_relaxed);
  uint64_t oth = other_packets_.load(std::memory_order_relaxed);
  uint64_t http = http_requests_.load(std::memory_order_relaxed);

  std::ostringstream oss;
  oss << Interval: packets= << ip << , bytes= << ib
      << | Total: packets= << tp << , bytes= << tb
      << | proto: TCP= << tcp << UDP= << udp << OTHER= << oth
      << | HTTP(req)= << http;

  std::lock_guard<std::mutex> lk(mx_);

  if (top_talkers > 0) {
    auto topSrc = analytics_.top_src_talkers(top_talkers);
    auto topDst = analytics_.top_dst_talkers(top_talkers);
    oss << \n Top SRC talkers(bytes): ;
    for (const auto& t : topSrc) oss << t.ip << = << t.bytes << ;
    oss << \n Top DST talkers(bytes): ;
    for (const auto& t : topDst) oss << t.ip << = << t.bytes << ;
  }

  if (top_flows > 0) {
    auto flows = analytics_.top_flows(top_flows);
    oss << \n Top Flows(bytes):;
    for (const auto& f : flows) {
      std::string proto = OTH;
      if (f.key.proto == FlowProto::TCP) proto = TCP;
      else if (f.key.proto == FlowProto::UDP) proto = UDP;
      oss << \n << proto <<
          << f.key.a.ip << : << f.key.a.port
          << <->
          << f.key.b.ip << : << f.key.b.port
          << bytes= << f.bytes
          << pkts= << f.packets;
    }
  }

  if (top_hosts > 0) {
    auto hosts = analytics_.top_http_hosts(top_hosts);
    oss << \n Top HTTP Hosts(hits): ;
    for (const auto& h : hosts) oss << h.host << = << h.hits << ;
  }

  return oss.str();
}

```

```
} // namespace npa
```

7.11 src/main.cpp (Orchestration, Threads, Timers, Pipeline)

Purpose: Wire together capture, parsing, reporting, and shutdown; keep capture non-blocking via async worker pool.

Listing 18: src/main.cpp (annotated)

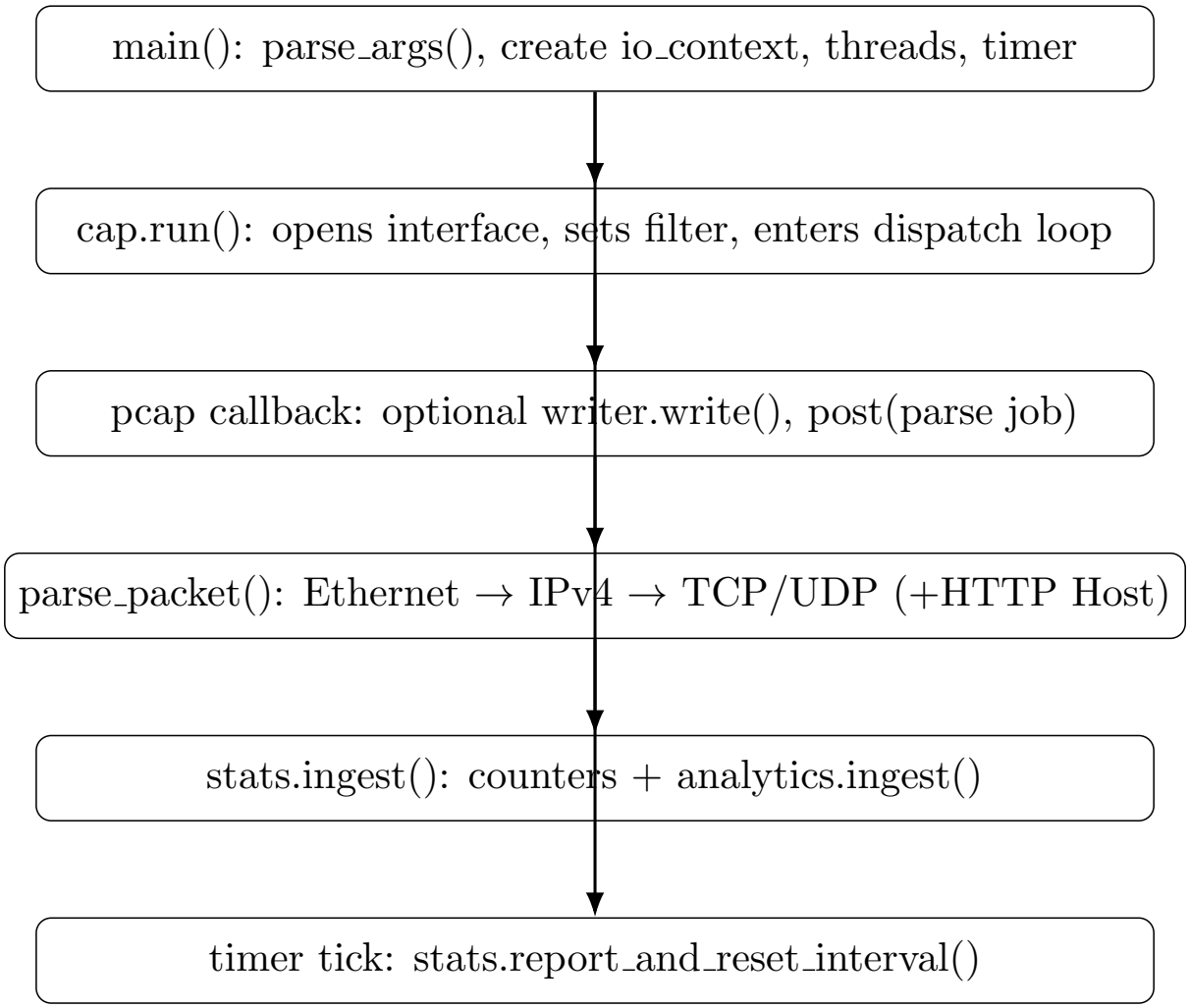
```
#include npa/config.hpp
#include npa/pcap_capture.hpp
#include npa/pcap_writer.hpp
#include npa/parser.hpp
#include npa/stats.hpp

#include <boost/asio.hpp>
#include <chrono>
#include <iostream>
#include <thread>
#include <vector>

static uint64_t now_ms() {
  using namespace std::chrono;
  return duration_cast<milliseconds>(steady_clock::now().time_since_epoch()).count();
}

int main(int argc, char** argv) {
  using namespace npa;

  Config cfg = parse_args(argc, argv);

  boost::asio::io_context io;
  Stats stats;

  boost::asio::steady_timer timer(io);
  std::function<void()> tick;
  tick = [&]() {
    timer.expires_after(std::chrono::seconds(cfg.stats_every_sec));
    timer.async_wait([&](const boost::system::error_code& ec) {
      if (!ec) {
        std::cout << stats.report_and_reset_interval(cfg.top_talkers, cfg.top_flows,
    ↪ cfg.top_hosts) << \n;
        tick();
      }
    });
  };
  tick();

  const int workers = std::max(1u, std::thread::hardware_concurrency());
  std::vector<std::thread> pool;
  pool.reserve(workers);
  for (int i = 0; i < workers; i++) {
    pool.emplace_back([&]() { io.run(); });
  }
```

```

  PcapCapture cap;
  PcapWriter writer;

  std::thread cap_thread([&]() {
    try {
      cap.run(cfg.iface, cfg.filter, cfg.promisc, cfg.snaplen,
        [&](const PcapPacket& p) {

          if (!cfg.pcap_out.empty() && !writer.enabled() && cap.native_handle() !=
    ↪ nullptr) {
            try {
              writer.open(reinterpret_cast<pcap_t*>(cap.native_handle()), cfg.pcap_out
    ↪ );
              std::cerr << PCAP output enabled:  << cfg.pcap_out << \n;
            } catch (const std::exception& e) {
              std::cerr << PCAP output error:  << e.what() << \n;
            }
          }

          if (writer.enabled()) {
            writer.write(reinterpret_cast<const pcap_pkthdr*>(p.pcap_hdr),
                         reinterpret_cast<const u_char*>(p.data));
          }

          auto buf = std::make_shared<std::vector<uint8_t>>(p.data, p.data + p.caplen)
    ↪ ;

          boost::asio::post(io, [buf, p, &stats, cfg]() {
            auto parsed = parse_packet(buf->data(), p.caplen, p.wirelen);
            if (!parsed) return;

            stats.ingest(*parsed, now_ms());

            if (cfg.verbose) {
              const auto& x = *parsed;
              std::cout
                << (x.l4 == L4Proto::TCP ? TCP : (x.l4 == L4Proto::UDP ? UDP : OTH))
                <<   << x.src_ip << : << x.src_port
                <<  ->  << x.dst_ip << : << x.dst_port
                <<  len= << x.wirelen;

              if (x.http) {
                std::cout <<  | HTTP  << x.http_method <<   << x.http_uri;
                if (!x.http_host.empty()) std::cout <<  Host= << x.http_host;
              }
              std::cout << \n;
            }
          });
        }
      );
    } catch (const std::exception& e) {
      std::cerr << Capture error:  << e.what() << \n;
    }
```

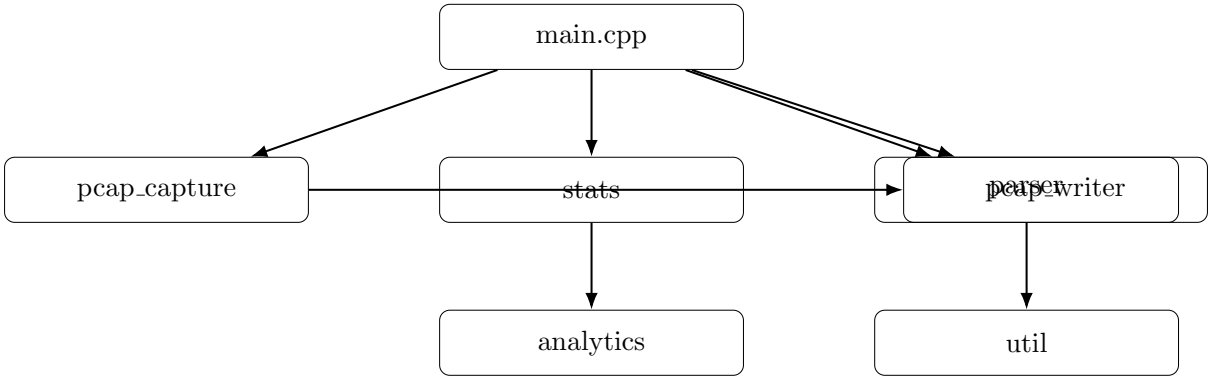
```
  });

  std::cout << Capturing on  << cfg.iface <<  with filter:  << cfg.filter << \n;
  if (!cfg.pcap_out.empty()) std::cout << PCAP output:  << cfg.pcap_out << \n;
  std::cout << Press ENTER to stop...\n;
  std::cin.get();

  cap.stop();
  io.stop();

  cap_thread.join();
  writer.close();
  for (auto& t : pool) t.join();

  return 0;
}
```

8 Extra Diagrams for Interviews

8.1 Call Graph



8.2 Dependency Map



9 Notes and Limitations (Honest Engineering)

- HTTP Host extraction works only for **plaintext HTTP**. HTTPS encrypts headers, so host is not visible without TLS parsing.
- TCP stream reassembly is not implemented. If an HTTP request spans multiple TCP segments, Host might not be extracted.
- IPv6 parsing is not implemented (extend in `parser.cpp` to support Ethertype 0x86DD).
- Flow eviction/timeouts are not implemented (long-running captures can grow memory usage; add idle timeouts to `Analytics`).

10 Future Work (Resume-Heavy Extensions)

- TLS SNI extraction (HTTPS host visibility)
- IPv6 parsing
- JSON export for dashboards
- Flow eviction + idle timeout
- ncurses terminal UI
- PCAP replay mode (offline analysis)
- Signature-based detection rules (basic IDS features)

11 Conclusion

This report included:

- complete folder + file mapping,
- full source code for each file as code blocks,
- line-by-line explanations via inline comments,

- architecture + call graph + dependency diagrams in TikZ,
- algorithms section (flow hashing + top-N selection),
- engineering notes, limitations, and future enhancements.